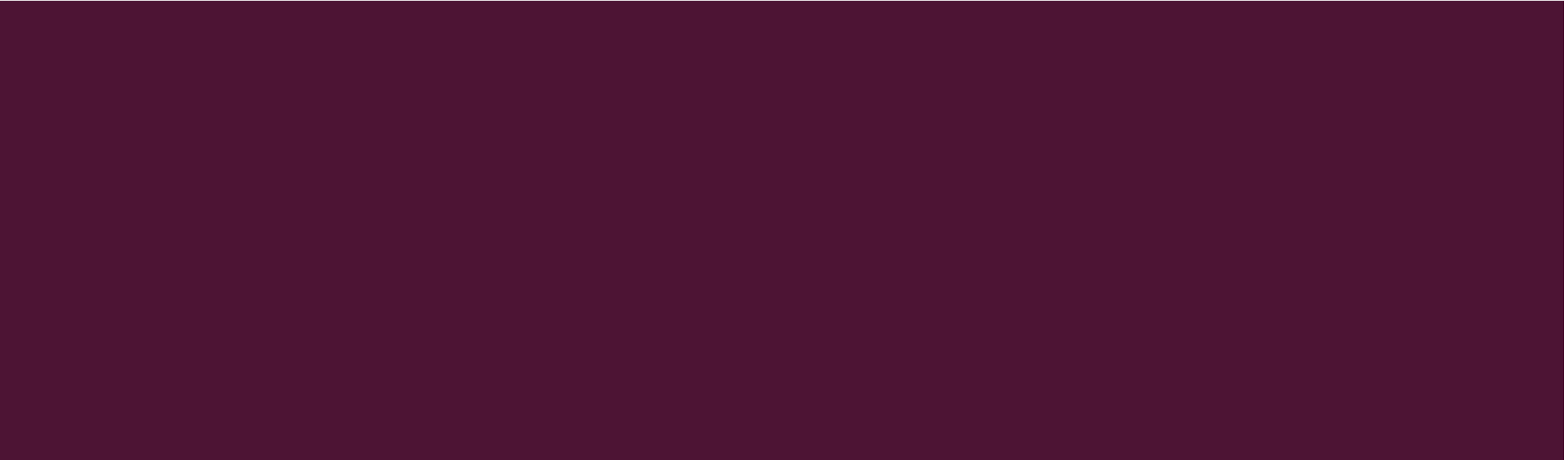




ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 5 – PATRONES DE DISEÑO (I)



¿QUÉ ES UN PATRÓN DE DISEÑO?

Descripción de un problema que ocurre repetidamente junto con la solución a ese problema

(Christopher Alexander)

ELEMENTOS DE UN PATRÓN DE DISEÑO

Nombre	Para aumentar nuestro “vocabulario” de diseño
Problema	Cuándo aplicar el patrón, describiendo el problema y el contexto
Solución	Elementos que forman el diseño junto con sus relaciones, responsabilidades y colaboraciones. Describe una plantilla
Consecuencias	Ventajas y desventajas de la aplicación de dicho patrón

DESCRIPCIÓN DE LOS PATRONES DE DISEÑO

- Nombre y clasificación
- Propósito
- También conocido como
- Motivación
- Aplicabilidad
- Estructura
- Participantes
- Colaboraciones
- Consecuencias
- Implementación
- Código de Ejemplo
- Usos conocidos
- Patrones relacionados

EL CATÁLOGO

PROPOSITO

PROPOSITO				
		Creación	Estructurales	Comportamiento
Ámbito	Clase	Factory Method	Adapter (de clases)	Interpreter Template Method
	Objeto	Abstract Factory Builder Prototype Singleton	Adapter (de objetos) Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

EL CATÁLOGO

Propósito	Patrones de diseño	Aspectos que pueden variar
De creación	Abstract Factory	La familia de objetos producidos
	Builder	Cómo se crea un objeto compuesto
	Factory Method	La subclase del objeto que es instanciado
	Prototype	La clase del objeto que es instanciado
	Singleton	La única instancia de una clase
Estructurales	Adapter	La interfaz de un objeto
	Bridge	La implementación de un objeto
	Composite	La estructura y composición de un objeto
	Decorator	Las responsabilidades de un objeto sin usar herencia
	Facade	La interfaz de un subsistema

EL CATÁLOGO

Propósito	Patrones de diseño	Aspectos que pueden variar
Estructurales	Flyweight	El coste de almacenamiento de los objetos
	Proxy	Cómo se accede a un objeto, su ubicación
De comportamiento	Chain of Responsibility	El objeto que puede satisfacer una petición
	Command	Cuándo y cómo se satisface una petición
	Interpreter	La gramática e interpretación de un lenguaje
	Mediator	Qué objetos interactúan entre si, y cómo
	Memento	Qué información privada se almacena fuera de un objeto, y cuándo
	Observer	El número de objetos que dependen de otro, cómo se mantiene actualizado el objeto dependiente
	State	El estado de un objeto

EL CATÁLOGO

Propósito	Patrones de diseño	Aspectos que pueden variar
De comportamiento	Strategy	Un algoritmo
	Template Method	Los pasos de un algoritmo
	Visitor	Las operaciones que pueden aplicarse a objetos sin cambiar sus clases

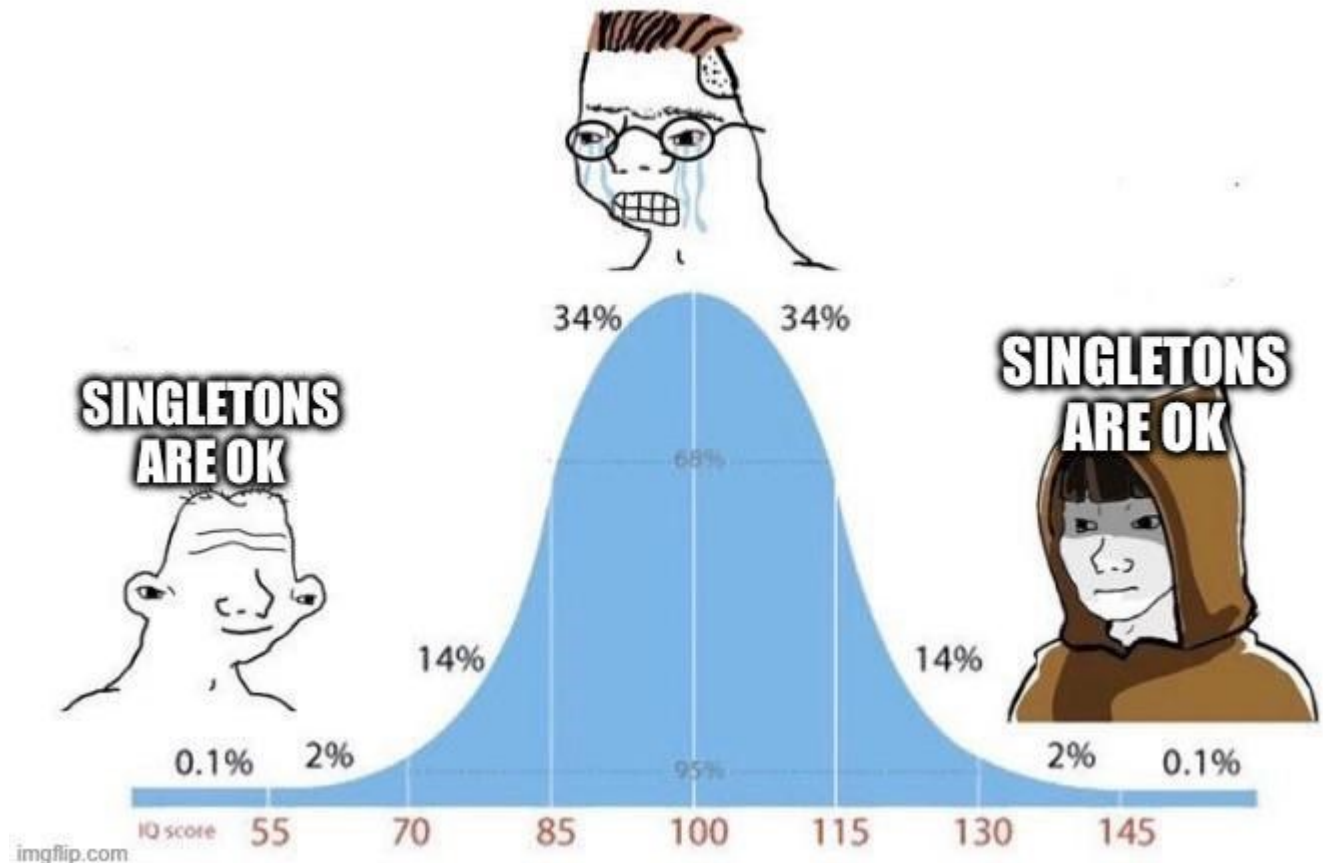
CREACIONAL SINGLETON

```
PUBLIC STATIC SINGLETON GETINSTANCE ( )  
{  
    IF (INSTANCE == NULL)  
        CREATEINSTANCE ( );  
  
    RETURN INSTANCE;  
}
```



quickmeme.com

SINGLETONS ARE BAD



CREACIONAL

SINGLETON

Propósito	Garantiza que una clase tenga una sola instancia, y proporciona un punto de acceso global a ella
También conocido como	Singular, Único
Motivación	• Se desea que una clase tenga exactamente una sola instancia, de fácil acceso
Aplicabilidad	• Se desea que exista una única instancia de una clase que sea accedida desde un punto conocido • Se desea que esta única instancia sea extensible mediante herencia

CREACIONAL

SINGLETON

Estructura

class Singleton

Singleton

- instanciaUnica
- singletonDatos

- + GetDataSingleton()
- + GetInstance()
return uniqueInstance
- + OperacionSingleton()

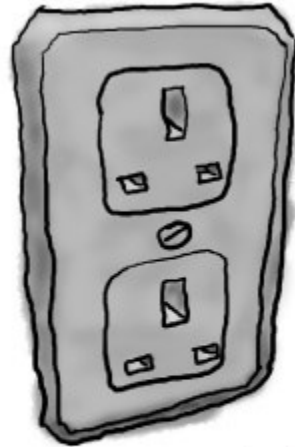
CREACIONAL

SINGLETON

Participantes	Singleton
Colaboraciones	Los clientes acceden a la instancia única solamente a través de la operación getInstance.
Consecuencias	• Acceso controlado a la única instancia • Espacio de nombres reducido • Permite el refinamiento de operaciones y la representación • Permite un número variable de instancias • Mas flexibles que las operaciones de clase

ESTRUCTURAL ADAPTER

British Wall Outlet



The British wall outlet exposes one interface for getting power.

AC Power Adapter



Standard AC Plug



The US laptop expects another interface.

The adapter converts one interface into another.

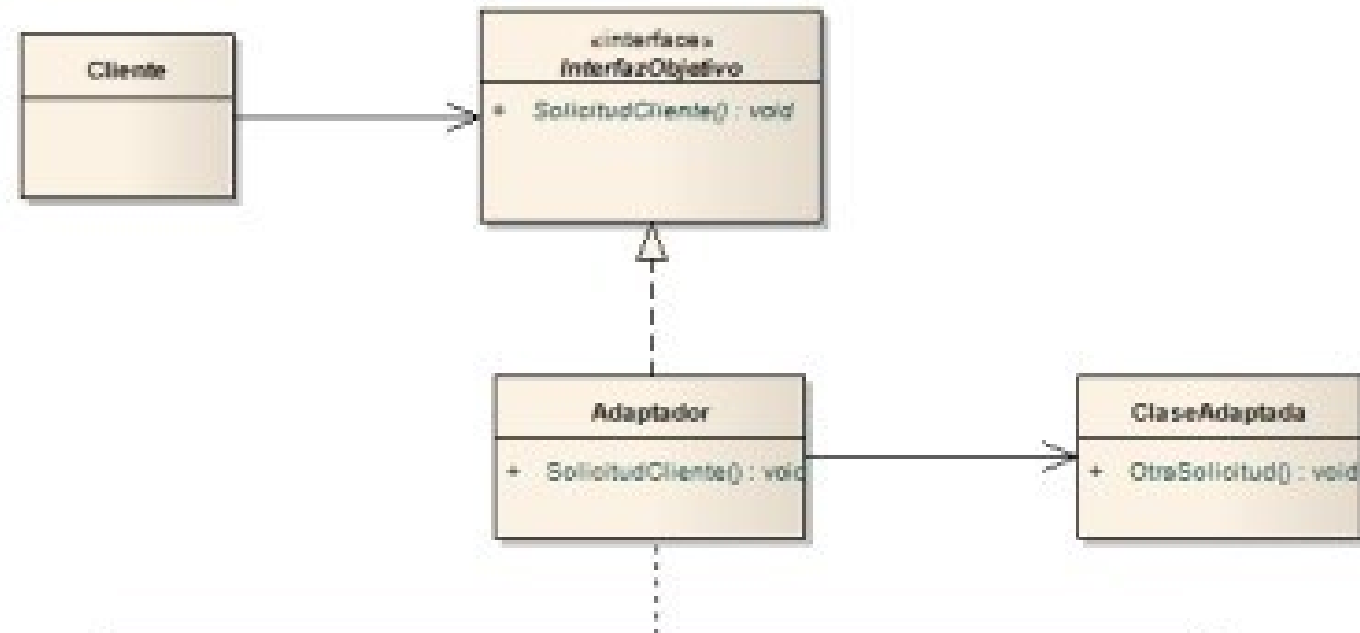
ESTRUCTURAL

ADAPTER

Propósito	Convierte la interfaz de una clase en otra interfaz que es la que esperan los clientes. Permite que cooperen clases que de otra forma no podrían por tener interfaces incompatibles
También conocido como	Wrapper (Envoltorio)
Motivación	• Una clase diseñada para reutilizarse no puede hacerlo porque su interfaz no coincide con la interfaz específica del dominio que requiere la aplicación
Aplicabilidad	• Se quiere usar una clase existente y su interfaz no concuerda con la que necesita • Se quiere crear una clase reutilizable que coopere con clases no relacionadas o que no han sido previstas (no tienen interfaces compatibles) • (Adapter de objetos) es necesario usar varias subclases existentes pero no es práctico adaptar su interfaz heredando de cada una de ellas

ESTRUCTURAL ADAPTER

Estructura



La clase Adaptador:

- Implementa la interfaz usada por Cliente.
- Incluye una referencia a ClaseAdaptada, pasada a través del constructor.
- Invoca los métodos de ClaseAdaptada en los métodos pertenecientes a la interfaz.
- Por ejemplo, `SolicitudCliente()` invocará a `OtraSolicitud()`

ESTRUCTURAL

ADAPTER

Participantes	Objetivo Cliente Adaptable Adaptador
Colaboraciones	La clase <i>Cliente</i> llama a las operaciones sobre una instancia <i>Adaptador</i> . Posteriormente, el <i>Adaptador</i> llama a las operaciones de <i>Adaptable</i> que llevan a cabo el pedido
Consecuencias	Adaptador de clase • Adapta una clase <i>Adaptable</i> a <i>Objetivo</i>, pero se refiere únicamente a una clase <i>Adaptable</i> concreta • Permite que <i>Adaptador</i> redefina parte del comportamiento de <i>Adaptable</i> por ser subclase • Introduce un solo objeto Adaptador de objeto • Permite que un mismo adaptador funcione con muchos <i>Adaptables</i> • Hace que sea más difícil redefinir el comportamiento de <i>Adaptable</i>

COMPORTAMIENTO STRATEGY



COMPORTAMIENTO

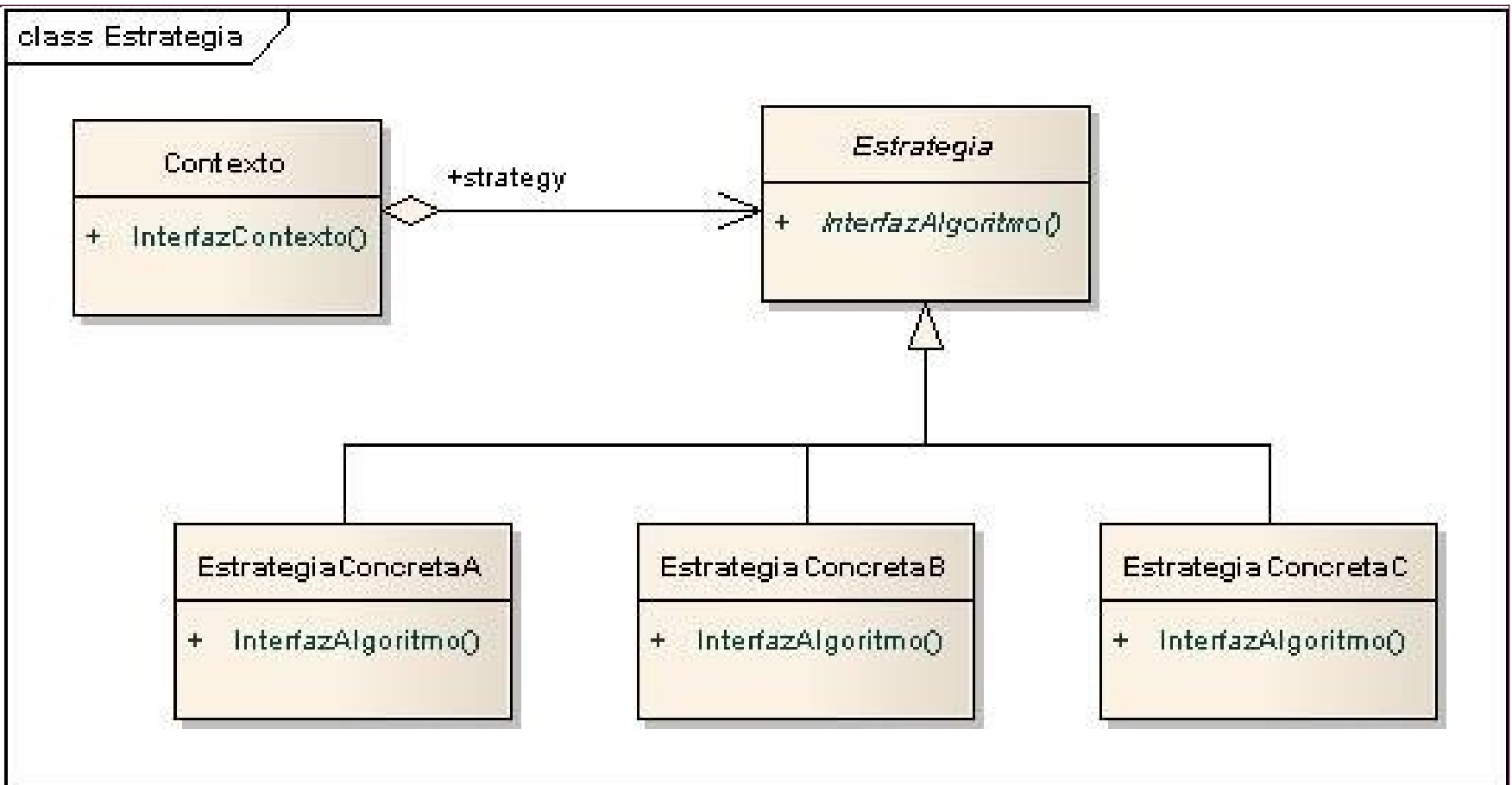
STRATEGY

Propósito	Define una familia de algoritmos, los encapsula a cada uno y los hace intercambiables. Permite que un algoritmo varíe independientemente de los clientes que lo usan
También conocido como	Policy (Política)
Motivación	• Un objeto debe hacer lo mismo pero con un algoritmo distinto o de manera distinta • La misma acción puede hacerse de distintas maneras en el objeto • Se necesita configurar el algoritmo que utilizará el objeto en tiempo de ejecución
Aplicabilidad	• Varias clases relacionadas que solo difieren en su comportamiento • Se necesitan variantes de un mismo algoritmo • Un algoritmo usa datos que un cliente no debe conocer. • Una clase define muchos comportamientos basados en condicionales

COMPORTAMIENTO

STRATEGY

Estructura



COMPORTAMIENTO STRATEGY

Participantes	Contexto Estrategia EstrategiaConcreta
Colaboraciones	<i>Estrategia</i> y <i>Contexto</i> interactúan para implementar el algoritmo elegido. <i>Contexto</i> envía todos los datos necesarios para que <i>Estrategia</i> decida qué algoritmo es llamado <i>Contexto</i> es la única interfaz que usan los clientes
Consecuencias	• Familia de algoritmos relacionados • Una alternativa a la herencia • Las estrategias eliminan las sentencias condicionales • Una elección de implementaciones • Los clientes deben conocer distintas estrategias • Costes de comunicación entre <i>Estrategia</i> y <i>Contexto</i> • Mayor número de objetos

BONUS TRACK:

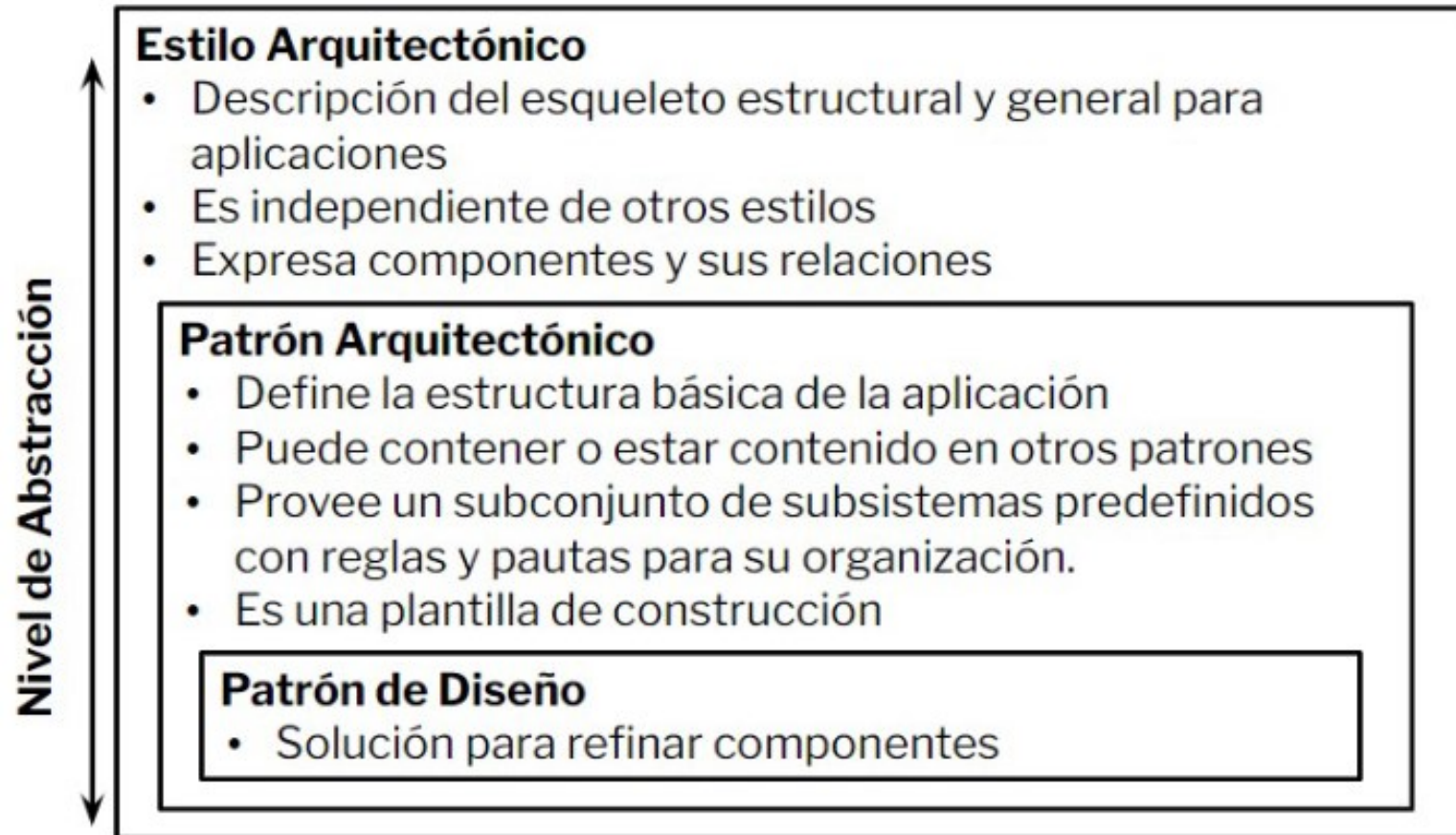
BREVE INTRODUCCIÓN A LAS ARQUITECTURAS DE SOFTWARE

“La arquitectura de software de un sistema es el **conjunto de estructuras** necesarias para razonar sobre el sistema, que comprende **elementos** de software, **relaciones** entre ellos y **propiedades** de ambos”

(Software Architecture in Practice)

BREVE INTRODUCCIÓN A LAS ARQUITECTURAS DE SOFTWARE

PATRONES ARQUITECTÓNICOS



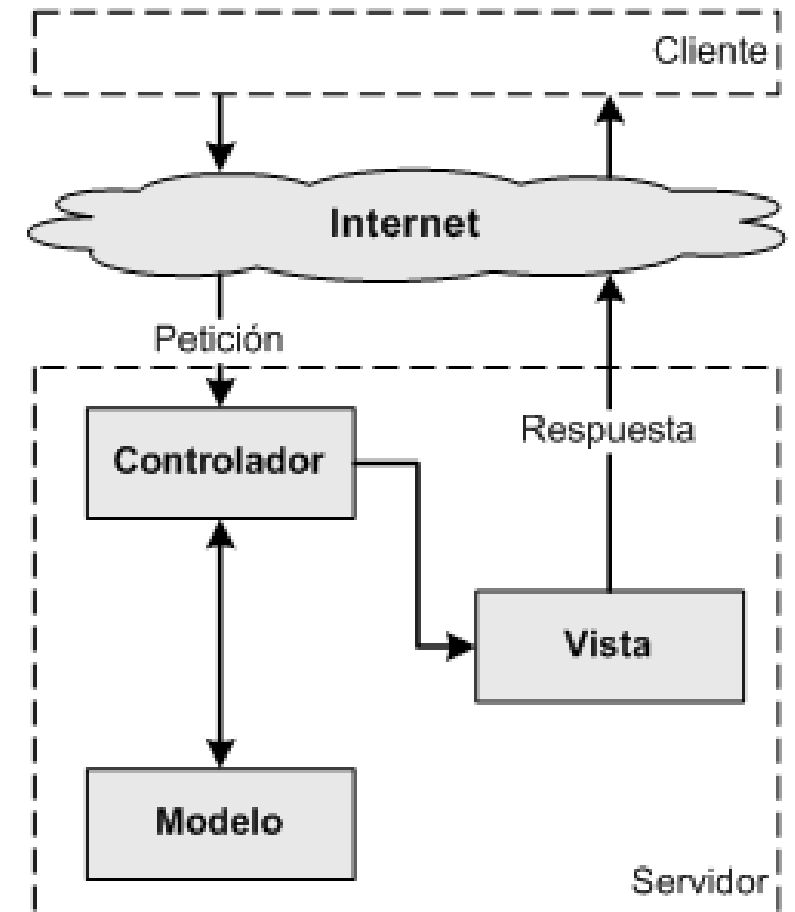
BREVE INTRODUCCIÓN A LAS ARQUITECTURAS DE SOFTWARE

PATRONES ARQUITECTÓNICOS: CATEGORÍAS

- Patrones simples
 - Layers
 - Pipes and filters
 - Blackboard
 - Repository
- Sistemas distribuidos
 - Broker
 - Client-Server
- Sistemas interactivos
 - Model-View-Controller
 - Presentation-Abstraction-Control
- Patrones adaptables
 - Microkernel

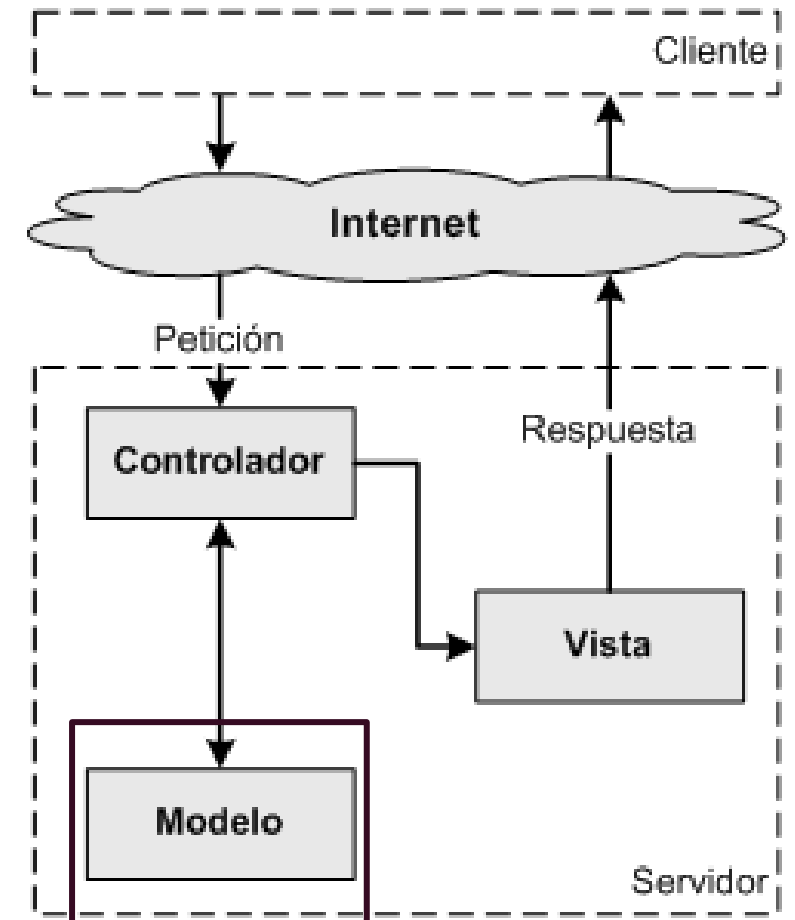
MODEL-VIEW-CONTROLLER (MVC)

- Es un patrón (inicialmente arquitectural) que guía acerca de cómo organizar y estructurar los componentes de un sistema
- Propone separar los componentes de un sistema en tres capas: modelo, vista y controlador
- Describe cómo se relacionan las tres capas, independientemente de la tecnología del sistema



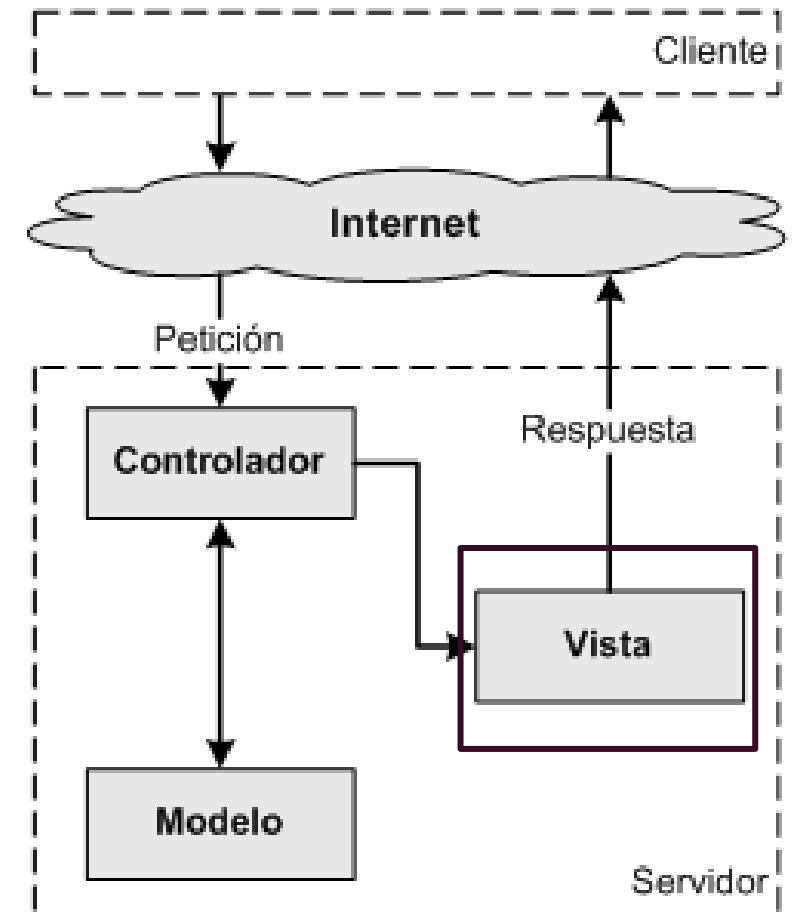
MODEL-VIEW-CONTROLLER

- En esta capa encontramos un conjunto de clases que representan las entidades del dominio
- Representa la lógica de la aplicación
- No debe conocer (teóricamente) a las vistas ni a los controladores
- Si necesitamos intercambiar datos con sistemas terceros encontraremos también a los DTOs.
- Se encarga también de manejar el almacenamiento y recuperación de entidades del dominio.



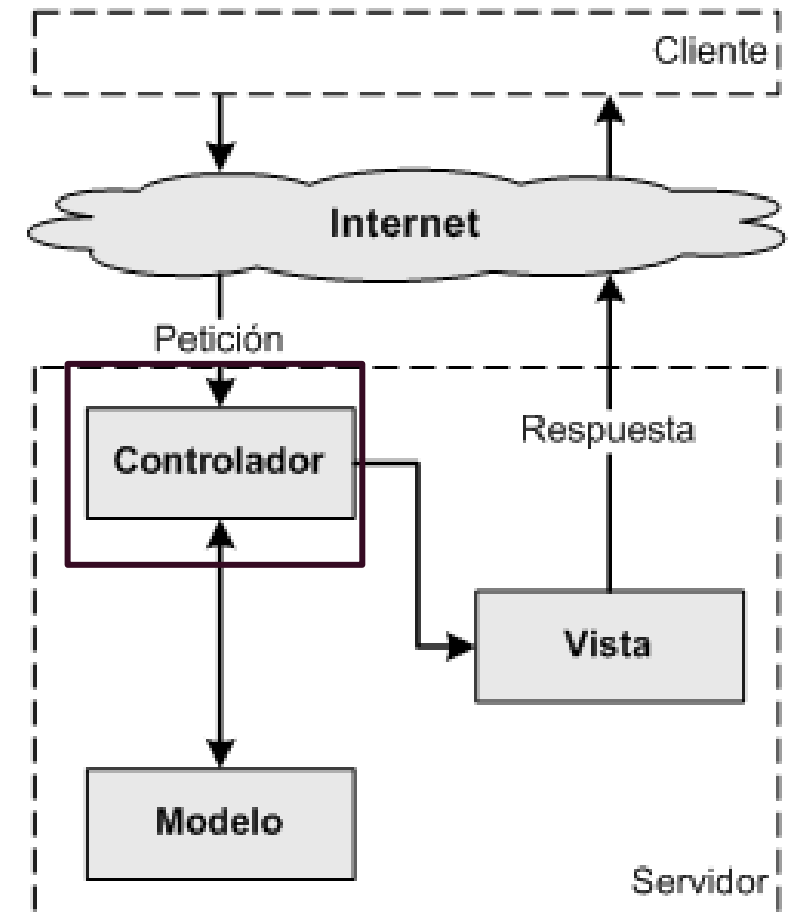
MODEL-VIEW-CONTROLLER

- En esta capa encontramos representaciones de los datos del modelo en un momento concreto para el usuario
- Generan la interfaz de la aplicación (cualquier tipo de dato utilizable por usuario o cliente del sistema)



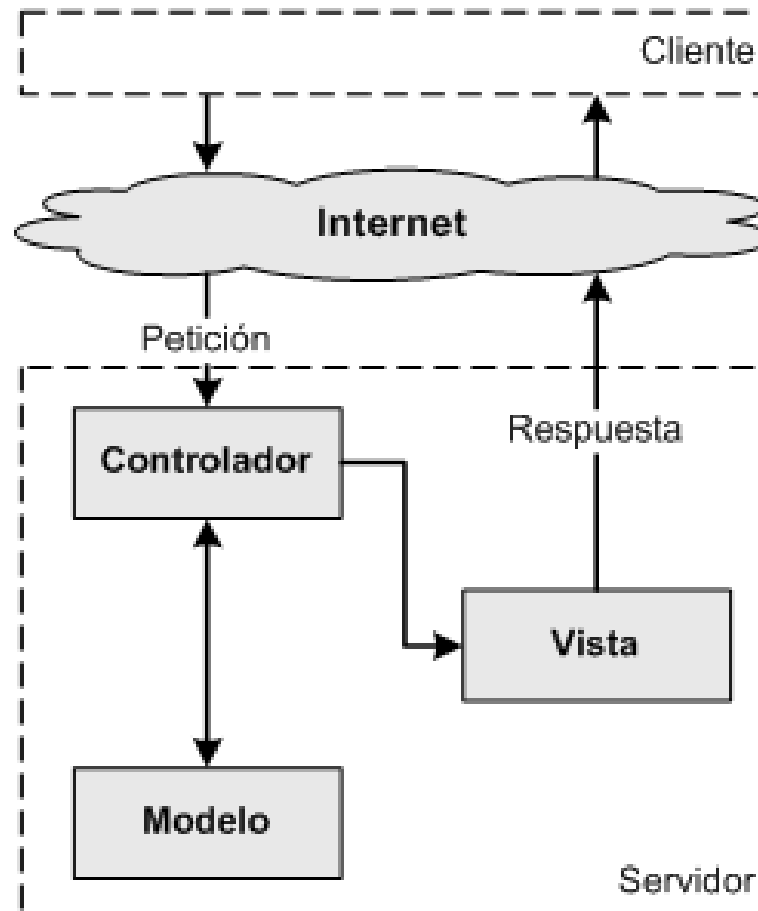
MODEL-VIEW-CONTROLLER

- Interpreta las peticiones del usuario y actúa en conformidad
- Es el intermediario entre el usuario y el sistema, y el Modelo y la Vista
- Transforman los datos entre el Modelo y la Vista
- Es un coordinador del sistema



MODEL-VIEW-CONTROLLER

¿DÓNDE VALIDAMOS LOS DATOS?



En todos lados (Modelo, Vista y Controlador)

MODEL-VIEW-CONTROLLER

VENTAJAS Y DESVENTAJAS

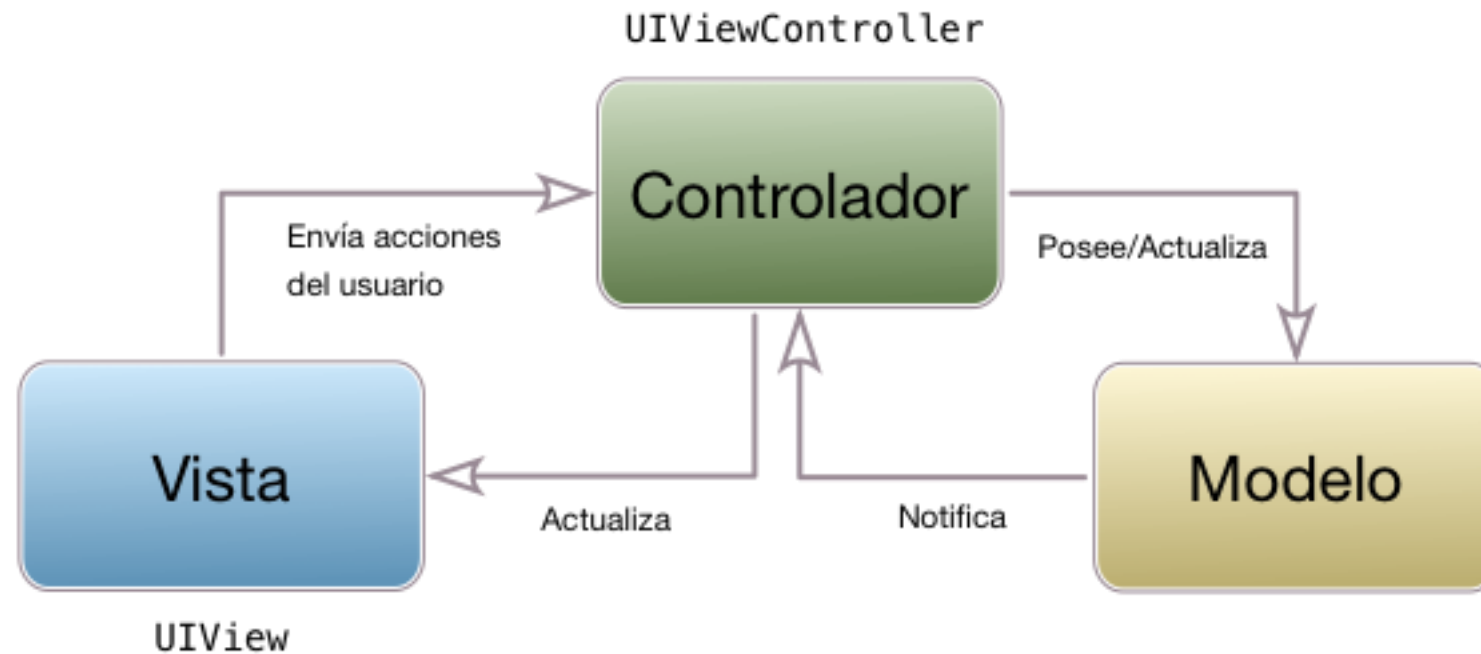
Ventajas

- Implementación modular
- Vistas muestran información actualizada
- Las modificaciones en el dominio solo afectan al Modelo
- Las modificaciones a las Vistas no afectan al modelo
- El testing es más fácil

Desventajas

- Mayor tiempo inicial de desarrollo (curva de aprendizaje)
- Hay que atenerse al patrón y sus convenciones, incluso para sistemas simples

MODEL-VIEW-CONTROLLER



BIBLIOGRAFÍA

- Gamma, E.; Helm, R.; Johnson, R. & Vlissides, J. M. (1994), *Design Patterns: Elements of Reusable Object-Oriented Software* , Addison-Wesley Professional .



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 5 – PATRONES DE DISEÑO (I)

